An Audit-Trail Framework for Machine-Learning Pipelines under SR 11-7 and the EU AI Act: A Demonstration on Volatility Forecasting

Akram Khan*

April 2026

Abstract

The U.S. Federal Reserve's SR 11-7 (April 2011) and the EU AI Act (Regulation 2024/1689, Articles 12 and 19) require firms deploying machine-learning models in regulated domains to maintain audit trails of every prediction. Code state, data state, model state, compute environment, and the prediction itself must all be recorded, with enough integrity protection that an external reviewer can verify any claim ex post.

Existing open-source ML tooling (MLflow, Weights & Biases, DVC, SageMaker Model Monitor) targets experiment tracking, model registry, or cloud-managed monitoring. None of these produce an SR 11-7-shaped audit trail with low-friction integration, vendor-neutral storage, and regulator-export capability out of the box. This paper introduces `mr-audit`, an open-source Python package that fills the gap.

The package writes one immutable, hash-chained record per prediction, capturing git commit hash, library versions, input hash, model parameters, compute environment, random seed, and prediction value. Records form a Merkle-style chain via SHA-256 hash linking, so tampering with any single record breaks the chain. The package ships with a replay engine, a drift-detection module (Kolmogorov-Smirnov plus Population Stability Index), and a regulator-export bundle that produces a self-contained zip whose integrity can be verified with off-the-shelf tools.

We demonstrate the package by retrospectively instrumenting the seven-model panel of Khan (2026c): S&P 500 5-day realized volatility, 980 trading days from 2022-01-03 to 2025-11-26. The instrumentation produces a 6,825-record audit log.

*Independent Researcher. ORCID: 0009-0002-7521-8648. Replication code, audit logs, and audit script: github.com/ayk5511/mr-audit.

Four use cases are demonstrated end-to-end. (i) Reproducibility audit: 5/5 randomly sampled records replay bit-identically. (ii) Drift detection: the KS test flags drift in 44 of 44 evaluated months relative to the 2022 Q1 baseline, consistent with the documented regime shift between high-vol 2022 and lower-vol 2023+. (iii) Per-model validation report generated from the log alone. (iv) Regulator-export bundle (841 KB zip; all SHA-256 checks pass).

The package, the demonstration scripts, the 6,825-record audit log, and an audit script that re-derives every numerical claim in this paper from JSON outputs are released at github.com/ayk5511/mr-audit, scoring 2 on the Reproducibility Disclosure Score rubric of Khan (2026a).

Keywords: model risk management, SR 11-7, OCC 2011-12, EU AI Act, audit trail, machine learning, reproducibility, compliance, regulatory technology, volatility forecasting

JEL Classification: C52, C53, C88, G17, G28

Contents

1 Introduction

A bank that deploys a machine-learning model into a decision-relevant role is, in the United States since 2011 and in the European Union since 2024, expected to maintain a record of every prediction the model produces: what code computed it, on what data, with what hyperparameters, in what compute environment, with what random seed, at what time. The expectations are codified in Federal Reserve SR 11-7 / OCC Bulletin 2011-12 (Federal Reserve, 2011; Office of the Comptroller of the Currency, 2011) and in the EU AI Act (European Parliament and Council, 2024), Articles 12 and 19. The compliance question is binary: either the audit trail exists, is intact, and is replayable, or it does not.

The technical question is more subtle. The compliance literature describes *what* should be in an audit trail; it says little about *how* to actually log one for a Python ML pipeline at scale. Existing open-source tools fall into three categories. (1) Experiment-tracking platforms (MLflow, Weights & Biases) are designed for the train-time exploration phase, with rich UIs for comparing runs but limited support for inference-time logging at scale and no built-in immutability or hash-chained integrity. (2) Data and model versioning tools (DVC, Pachyderm) version artifacts but do not log per-prediction inputs and outputs. (3) Cloud-managed monitoring services (AWS SageMaker Model Monitor, Google Vertex AI Model Monitoring) are vendor-locked, expensive at audit-grade volume, and require trusting the cloud provider with the exact inference data the audit is designed to protect.

What is missing is an open-source, vendor-neutral Python package that produces an SR 11-7-shaped audit trail with low-friction integration. We provide one and demonstrate it.

1.1 Contribution

This paper makes two contributions.

First, we introduce `mr-audit`, an open-source Python package (MIT-licensed; PyPI `mr-audit`; repository github.com/ayk5511/mr-audit) that produces immutable, hash-chained, replayable, drift-aware audit trails for ML inference pipelines. The package is integrated into a calling pipeline by a one-line decorator (`@auditable`) and a context manager (`AuditContext`); no central server, no cloud account, no schema-design effort. Records are stored locally in either SQLite or JSON Lines format, both vendor-neutral. The package implements a SHA-256 hash chain (each record's identifier is the canonical-JSON SHA-256 of its content; each record carries the previous record's identifier; tampering with any single record breaks the chain). It ships with a replay engine that reconstructs any prediction from the log, a drift-detection module (Kolmogorov-Smirnov two-sample test plus Population Stability Index with Laplace smoothing), and

a regulator-export bundle that produces a self-contained zip archive whose integrity can be verified using off-the-shelf hash tools without any `mr-audit` code installed.

Second, we demonstrate the package on a real ML pipeline by retrospectively instrumenting the seven-model volatility-forecasting panel of Khan (2026c) (S&P 500 5-day realized volatility forecasts from three GARCH variants, HAR-RV, LightGBM, XGBoost, and an equal-weight ensemble across 980 trading days, 2022-01-03 to 2025-11-26). The same panel is the empirical demonstration of Khan (2026b), which uses `vol-eval` to apply Diebold-Mariano, Model Confidence Set, and Hansen SPA tests to the seven models. `mr-audit` and `vol-eval` are intended as complementary infrastructure: `vol-eval` addresses the question "did your forecasting comparison use the right significance test?"; `mr-audit` addresses "can you prove tomorrow that today's prediction was made with this code, this data, and these parameters?". The instrumented run produces a 6,825-record audit log. We then exercise the four primary use cases an SR 11-7 model risk management team would run on such a log:

1. **Reproducibility audit.** A randomly-sampled set of 5 records replays bit-identically: the original input hash, the model parameters, and the final prediction match exactly when re-run.

2. **Drift detection.** The KS test flags drift in 44 of the 44 monthly windows evaluated against the 2022 Q1 baseline. The pattern is consistent with the documented regime shift between high-vol 2022 and lower-vol 2023+ in Khan (2026c).

3. **Validation report.** Per-model summary statistics (n, prediction mean, prediction std, library versions at last record) are computable from the audit log alone, with no external data.

4. **Regulator export.** A 841 KB zip bundle is produced from the 6,825-record log; its manifest, file SHA-256 hashes, and chain integrity all verify cleanly using a stand-alone tool that imports nothing from `mr-audit`.

1.2 What this paper claims and does not claim

Three explicit scoping decisions deserve flagging.

First, we do *not* claim that using `mr-audit` makes any model *compliant* with SR 11-7, OCC 2011-12, or the EU AI Act. Compliance is a determination by the firm and its supervisor; the package provides the technical record-keeping infrastructure that compliance requires.

Second, the empirical demonstration is a *retrospective* instrumentation. The 6,825 records are written today against the previously-fit forecast panel of Khan (2026c); we are not running the seven-model pipeline live and recording in real time. The retrospective

form is appropriate for the demonstration's purpose (showing that the four use cases work end-to-end at a realistic scale, ∼7,000 records) but does not establish that `mr-audit` would handle a live high-throughput inference workload without further engineering. Section 5.3 discusses this honestly.

Third, the EU AI Act's high-risk classification (Annex III) does not currently include volatility forecasting. SR 11-7 is the live regulatory pull for the demonstration's empirical target; the EU AI Act framing is forward-looking. We do not overclaim that vol forecasting is presently in scope of the AI Act's high-risk regime.

1.3 Roadmap

Section 2 reviews the SR 11-7 and EU AI Act requirements relevant to audit trails, distinguishing what the regulations specify from what they leave to the firm. Section 3 describes the `mr-audit` package: design principles, schema, hash chain, storage backends, replay engine, drift module, export bundle, and CLI. Section 4 reports the empirical demonstration on the Khan (2026) volatility panel, including the four use cases. Section 5 discusses the implications and limitations. Section 6 states the reproducibility commitment, including the audit script that re-derives every numerical claim in this paper.

2 Regulatory framework

The audit-trail requirements an ML pipeline must satisfy come from three regulatory sources of growing weight: SR 11-7 / OCC 2011-12 (United States, 2011), the EU AI Act (Regulation 2024/1689, 2024), and the NIST AI Risk Management Framework (NIST AI 100-1, 2023). They differ in legal force and in granularity but converge on a common set of technical record-keeping requirements that the package described in Section 3 aims to satisfy.

2.1 SR 11-7 / OCC 2011-12

Federal Reserve (2011) and the companion Office of the Comptroller of the Currency (2011) are the canonical U.S. supervisory guidance on model risk management for federally-regulated banking institutions. The guidance applies to "models" broadly defined, encompassing both traditional statistical models and modern machine-learning systems. The relevant requirements for audit trails fall in three sections of the guidance:

Model documentation (SR 11-7 ğV.G). Banks are expected to document each model's purpose, theory, methodology, assumptions, and limitations, with sufficient detail that a knowledgeable reader unfamiliar with the model can understand its operation. For ML models, this includes the choice of architecture, hyperparameters, training data, and validation results.

Ongoing monitoring (SR 11-7 ğIV.C, within Model Validation). Banks are expected to verify that models perform as intended over time, with specific attention to changes in input data distributions, model drift, and the continued validity of model assumptions. SR 11-7 explicitly mentions monitoring of input variables for drift relative to the development sample.

Model inventory (SR 11-7 ğV.F). Each model in production must be in a model inventory that captures (at minimum) the model identity, version, owner, validation status, and use cases. The inventory is the durable record an examiner consults during a supervisory review.

The guidance is principles-based; it does not specify how the documentation, monitoring, or inventory should be technically implemented. In practice, large U.S. banks operate model risk management organizations of 50 to several hundred staff who maintain these records, often through a combination of centralized governance platforms (e.g., SAS Model Manager) and bespoke spreadsheet-based tracking.

2.2 The EU AI Act

European Parliament and Council (2024) (Regulation (EU) 2024/1689) entered into force on August 1, 2024 with phased implementation: prohibited AI practices became applicable on February 2, 2025, obligations for general-purpose AI models on August 2, 2025, and most remaining obligations including the high-risk regime on August 2, 2026.

For "high-risk AI systems" (defined in Article 6 with reference to Annexes I and III), the Act imposes specific technical record-keeping requirements that are stricter than SR 11-7. Two articles are directly relevant.

Article 12 – Record-keeping. High-risk AI systems must technically allow for the automatic recording of events (logs) over the lifetime of the system. The logs must enable monitoring of operation in line with intended purpose and, in particular, identification of situations which may result in the AI system presenting a risk within the meaning of Article 79.

Article 19 – Automatically generated logs. Providers of high-risk AI systems must keep the logs automatically generated by their AI systems for a period determined by Union or national law, and at minimum for six months after the system is placed on the market. The logs must include, at minimum, the period of each use, the reference database against which input data has been checked, the input data for which the search has led to a match, and the identification of the natural persons involved in the verification of the results.

The volatility-forecasting use case demonstrated in Section 4 is not currently classified as "high-risk" under Annex III, which lists creditworthiness and access to essential services among financial-sector use cases but not market-data forecasting. The AI Act framing is

therefore forward-looking for the present demonstration; we record audit data that would satisfy Article 12 and Article 19 in the event that the use case were brought into scope, or in the event that the firm chooses to apply the same standard voluntarily.

Table 1 maps each Article 19 sub-requirement to the specific `mr-audit` fields that satisfy it, providing a compliance traceability matrix a CRO can hand to a regulator without re-engineering. The `cross-domain` examples in the package's `examples/` directory (a sklearn LogisticRegression credit-scorecard scenario; a statsmodels ARIMA macroeconomic forecaster) demonstrate the same field mapping holds for in-scope Annex III.5(b) credit-scoring use cases as well as for the demonstration's volatility forecaster.

Table 1: EU AI Act Article 19 sub-requirements mapped to `mr-audit` schema fields. The Article 19 sub-requirements listed here are the four explicit logging items in the regulation's text; "identification of natural persons" is the only one that requires firm-side application policy rather than automatic capture by the package.

Article 19 sub-requirement	`mr-audit` field(s)	Captured automatically?
Period of each use	`timestamp_utc`	yes (per record)
	`prediction.predicted_for_date`	yes (caller-supplied)
Reference database against which input data has been checked	`data.input_data_source`	yes (caller-supplied)
	`data.input_data_hash`	yes (auto-computed)
	`data.feature_names`	yes (from extractor)
Input data for which the search has led to a match	`data.feature_values_summary`	yes (from extractor)
	`data.input_data_hash`	yes (deterministic)
Identification of natural persons involved in verification of results	(firm-side application policy)	no – requires firm input
	e.g. `model.parameters["reviewer_id"]`	yes if firm passes it
Beyond Article 19, additional captures:		
Code provenance	`code.code_commit_hash`	yes (from git)
Library provenance	`code.library_versions`	yes (from `sys.modules`)
Model identity + parameters	`model.{name, version, parameters}`	yes
Compute environment	`compute.{python_version, platform}`	yes
Random state	`random_seed`	yes (caller-supplied)
Tamper-evidence	`prev_record_hash` chain	yes (auto-computed)

The two right-most columns separate concerns honestly: automatic capture (the package's responsibility) from caller-supplied data (the firm's responsibility, e.g. logging a reviewer's identifier when a human-in-the-loop is part of the AI system). The package does not attempt to enforce the latter; it provides the schema slot.

2.3 NIST AI Risk Management Framework

The NIST AI RMF (National Institute of Standards and Technology, 2023) is a voluntary U.S. framework structured around four functions: GOVERN (organisational policies and accountability), MAP (context and risk identification), MEASURE (analysis, assessment, and tracking), and MANAGE (prioritisation and response). The MEASURE function

explicitly references monitoring for distributional shifts in input data, behavioural drift in model outputs, and traceability of decisions. NIST AI 100-1 is not legally binding, but it is increasingly cited by firms as the de facto operational implementation framework that "operationalises" SR 11-7's principles for ML models.

2.4 Synthesis: what an audit trail must capture

Across the three frameworks, an audit trail for an ML inference pipeline should record, at the minimum, the following per-prediction state.

Table 2: Audit-trail field requirements derived from SR 11-7, EU AI Act Articles 12 and 19, and NIST AI 100-1. "✓" indicates the framework explicitly references the requirement; blank indicates the requirement is implicit in the broader guidance.

Field	SR 11-7	EU AI Act	NIST AI 100-1
Model identity (name, version)	✓	✓	✓
Model parameters	✓		✓
Model artifact identifier		✓	
Code state (commit, libraries)			✓
Input data identifier or hash	✓	✓	✓
Input feature values or summary	✓		✓
Compute environment			
Random seed (for stochastic models)			✓
Prediction value	✓	✓	✓
Prediction date / horizon	✓	✓	✓
Timestamp of prediction	✓	✓	✓
Integrity of the log over time	✓	✓	
Replayability (reconstruct prediction)	✓		✓
Drift detection (input + output)	✓	✓	✓

The package described in Section 3 captures every row of Table 2 by default, with the exception that input feature values are stored as a configurable summary (mean/std or full-vector at the firm's choice) rather than as the raw tick-level inputs, to keep log sizes tractable.

2.5 Existing tooling, with honest assessment

Several open-source and commercial tools partially address the requirements in Table 2. None, in our reading, provides a complete solution out of the box:

- **MLflow** (Databricks, OSS): rich experiment tracking and model registry, but designed for the train-time exploration phase. Inference-time logging is supported but lacks immutability, hash-chained integrity, and a regulator-export bundle.

- **Weights & Biases** (commercial, hosted): excellent UI; data leaves the firm's control; expensive at audit-grade volume; no compliance certification.
- **DVC** (Iterative.ai, OSS): data and code version control. No per-prediction inference logging.
- **Pachyderm** (Hewlett Packard Enterprise, OSS): data versioning and pipeline orchestration. Same gap as DVC for inference logging.
- **AWS SageMaker Model Monitor / Google Vertex AI Model Monitoring**: cloud-locked; not portable; the inference data sits on the cloud provider's infrastructure.
- **IBM AI FactSheets**: documentation templates, not runtime logging.

We do not claim our package is uniformly superior to these tools. MLflow has a far richer experiment-management UI, Weights & Biases has hosted infrastructure, DVC has stronger data versioning. Our claim is narrower: `mr-audit` is the open-source Python package most narrowly designed for the specific requirements of Table 2, with low integration friction and no vendor lock-in.

Table 3 summarises the comparison along the audit-trail dimensions that matter for SR 11-7 / EU AI Act compliance. The table is intentionally narrow in scope: it does not score the tools on dimensions where they are clearly stronger than `mr-audit` (e.g., MLflow's experiment-comparison UI, W&B's hosted dashboards, DVC's data versioning). Within the audit-trail dimensions, `mr-audit` is the only entry that satisfies all requirements out of the box.

Table 3: Audit-trail capability comparison across open-source and commercial tooling. ✓ = native support; • = partial / requires custom code; – = not provided. Cells reflect the tools' default behaviour as of v2024–2025 documentation review.

Capability	`mr-audit`	MLflow	W&B	DVC	SageMaker Monitor	IBM AI FactSheets
Per-prediction logging	✓	•	•	–	✓	–
Hash-chain integrity	✓	–	–	–	–	–
Replay engine (bit-identical)	✓	–	–	–	–	–
Input drift detection	✓	–	•	–	✓	–
Output drift detection	✓	–	•	–	✓	–
Regulator-export bundle	✓	–	–	–	–	•
Code-state provenance (commit, libs)	✓	•	•	✓	–	–
Vendor-neutral local storage	✓	✓	–	✓	–	✓
Open source (MIT/Apache)	✓	✓	–	✓	–	•

3 The mr-audit package

This section describes the design and implementation of `mr-audit`. The package is open-source under MIT license at github.com/ayk5511/mr-audit; the version described here is 0.0.1.

3.1 Design principles

The package follows four design principles, in order of priority.

1. Vendor-neutrality. All persistence is local. The two storage backends (SQLite, JSON Lines) are open formats with stable specifications. There is no cloud service, no central server, no schema-design platform, no proprietary file format. A firm can run `mr-audit` entirely on-premises, in a regulated network, with the audit log on a write-only file system.

2. Low integration friction. Adding `mr-audit` to an existing pipeline is one decorator and one context manager: the developer adds `@auditable(model_name=..., model_version=...)` to the prediction function, and wraps the calling code in `with AuditContext(log_path=...): ...`. Without an active context the decorator is a no-op (the function runs unrecorded), so the same code can be run with or without auditing depending on context.

3. Immutability and integrity. The log is append-only by API; there is no public "edit" or "delete" operation. Each record carries a SHA-256 identifier computed from its canonical-JSON content (excluding the identifier itself), and a pointer to the previous record's identifier. Tampering with any single record breaks the chain at that point.

4. Replayability. Every prediction must be reconstructable from the log alone, given the unwrapped prediction function. The replay engine recomputes the input hash from the supplied inputs and re-runs the function; both the input hash and the prediction value must match the recorded values for the replay to be considered valid.

3.2 Schema

Each audit record is a Python `AuditRecord` dataclass with the following structure (Table 4). The schema is versioned (`SCHEMA_VERSION = "0.1.0"`); future schema evolutions will increment this version while remaining backward-readable for older records.

Table 4: The `AuditRecord` schema. Substate fields are themselves dataclasses; the table shows the fully-flattened key set.

Field	Type	Captures
`schema_version`	str	e.g. "0.1.0"
`timestamp_utc`	str (ISO 8601)	when the prediction was logged
code (CodeState)		
`code_commit_hash`	str|None	git SHA of the calling repository
`code_dirty`	bool	uncommitted changes at predict time
`library_versions`	dict[str,str]	numpy, pandas, scipy, torch, sklearn, arch, statsmodels
data (DataState)		
`input_data_hash`	str (hex)	SHA-256 of canonical-JSON of inputs
`input_data_source`	str|None	free-form data source identifier
`feature_names`	tuple[str]	ordered feature names
`feature_values_summary`	dict|None	feature values or summary stats
model (ModelState)		
`name`	str	e.g. "GJR-GARCH"
`version`	str	e.g. "1.0.0"
`parameters`	dict[str,Any]	hyperparameter set
`model_artifact_hash`	str|None	SHA-256 of serialised model
compute (ComputeState)		
`python_version`	str	e.g. "3.14.0"
`platform`	str	OS + arch
`process_id`	int|None	PID of the predict call
prediction (Prediction)		
`value`	float | list[float]	scalar or vector
`horizon_days`	int|None	forecast horizon
`predicted_for_date`	str|None	ISO 8601 date
`random_seed`	int|None	seed used for stochastic models
`prev_record_hash`	str|None	SHA-256 of preceding record (chain link)
`record_id`	str	SHA-256 of this record's canonical JSON

The `record_id` is computed at write time by the storage backend, not by the user, and is excluded from its own hash input. The `prev_record_hash` of the first record in any log is `None` (the genesis record).

3.3 Hash chain

The hash chain implements a Merkle-style integrity guarantee. Each record's identifier is computed as

$$\texttt{record\_id} = \texttt{SHA-256}(\,\text{canonical-JSON}(R \setminus \{\texttt{record\_id}\})\,)$$

where canonical-JSON is the deterministic serialisation defined by `json.dumps(..., sort_keys=True, separators=(",", ":"))`. The exclusion of `record_id` from its own hash input is necessary; the exclusion is implemented in `compute_record_id` (`src/mr_audit/core/hash.py`).

Chain integrity is verified by `verify_chain`, which performs three checks for each record:

1. The record's `record_id` matches the canonical hash of its own contents.
2. The record's `prev_record_hash` matches the `record_id` of the immediately preceding record in insertion order.
3. The first record's `prev_record_hash` is `None`.

Tampering with any field of any record breaks check (1) for that record and all subsequent records (because the chain link cascades). Reordering records breaks check (2). Inserting a forged record requires both forging a valid hash for the inserted record and re-computing all subsequent records' hashes, which is computationally infeasible if the chain is also externally backed up at any point.

The chosen hash function is SHA-256 (NIST FIPS 180-4). The choice is consistent with current best practice and aligns with the EU AI Act's intent in Article 19 without specifying any particular algorithm.

3.4 Storage backends

Two backends are implemented, both vendor-neutral and locally runnable.

SQLiteStore (`src/mr_audit/core/store.py`). Records are stored in a single-table schema with columns `(id INTEGER PRIMARY KEY AUTOINCREMENT, record_id TEXT UNIQUE, prev_record_hash TEXT, timestamp_utc TEXT, json TEXT)`. Indexes on `record_id` and `timestamp_utc` provide $O(\log n)$ lookup by either. Suited for production use up to millions of records on a single file.

JSONLStore. Records are appended one per line to a single file. Lightweight, human-readable, easily processed by `jq`, `pandas`, or `R`. Suited for lightweight pipelines and human inspection. Random access requires a full scan; suited for moderate sizes.

Both backends preserve insertion order, which is what the hash chain depends on. Both are append-only by API. No public API exposes deletion or update.

3.5 Replay engine

The `replay` function in `src/mr_audit/replay/engine.py` reconstructs any prediction from a sealed audit record. The contract is: given the record and the unwrapped prediction function, calling the function with the recorded inputs must produce the recorded prediction. The function returns a `ReplayResult` dataclass with three principal flags:

- `input_hash_match` – the canonical-JSON hash of the supplied inputs equals the recorded `input_data_hash`.

- `prediction_match` – the re-run output equals the recorded `prediction.value` within the supplied tolerance (default 10^{-9}).
- `library_version_warnings` – non-fatal diagnostic; library versions present at replay time are compared to the versions recorded.

Replay is non-destructive: the input record is never mutated. The supplied inputs are re-hashed and compared; if the supplied inputs do not match the recorded input hash (Section 4.2), the result still re-runs the function but flags the mismatch for the auditor's attention.

3.6 Drift detection module

The `detect_drift` function in `src/mr_audit/drift/input.py` compares two record sequences (a baseline and a window) for distribution drift across the recorded feature summaries. Two methods are computed for each feature:

Kolmogorov-Smirnov two-sample test. Null hypothesis: the two samples come from the same distribution. Returns the test statistic and an asymptotic p-value. We use SciPy's `stats.ks_2samp` for the implementation.

Population Stability Index. A scalar measure of distribution shift, defined as

$$\mathrm{PSI} = \sum_{i=1}^{B} (p_{w,i} - p_{b,i}) \cdot \log\left(\frac{p_{w,i}}{p_{b,i}}\right),$$

where $p_{b,i}$ and $p_{w,i}$ are the proportions of baseline and window samples in bucket i, respectively. Buckets are equal-frequency on the baseline. Empty buckets are handled by Laplace (add-half) smoothing – adding 0.5 to each bucket count before normalising – to avoid log(0) blowups while keeping the estimator unbiased on well-populated buckets. Standard credit-risk thresholds (PSI < 0.10 stable, 0.10–0.25 moderate, ≥ 0.25 high) are widely adopted in bank model-risk practice.

The choice to compute drift on *summaries* rather than raw feature values is deliberate. Audit logs that store full feature vectors per prediction grow rapidly and may capture sensitive customer-level information. The default summary captures the per-feature value at prediction time but not the underlying customer record; firms can opt into full-vector storage by passing `summarise_features=False` to the `@auditable` decorator.

3.7 Regulator export bundle

The `export_bundle` function in `src/mr_audit/export/bundle.py` produces a self-contained zip archive that an external reviewer can verify without any `mr-audit` code installed. The bundle contains:

- `manifest.json` – bundle metadata + per-file SHA-256 hashes
- `audit_records.jsonl` – all records, canonical JSON, one per line
- `chain_verification.json` – chain verification result at export time
- `README.txt` – human-readable description and replay instructions
- `manifest.sha256` – SHA-256 of `manifest.json` in the standard `sha256sum`-compatible format

A reviewer with only `sha256sum` and a JSON parser can verify the bundle's integrity end-to-end: `sha256sum manifest.json` should match `manifest.sha256`; for each file in `manifest.json`'s `files` field, `sha256sum <name>` should match the listed digest. With `mr-audit` installed, the verification is one command: `mr-audit inspect <bundle>`.

3.8 Command-line interface

The `mr-audit` CLI provides four subcommands, each one operation a model risk team would execute from a terminal:

```
mr-audit show   <log>           # tabular summary of records
mr-audit verify <log>           # hash-chain integrity check
mr-audit export <log> <bundle>  # build regulator-ready bundle
mr-audit inspect <bundle>       # verify and summarise a bundle
```

The CLI is deliberately small. Programmatic users should import the Python API; the CLI is for one-off inspection and audit-handoff workflows.

3.9 Test coverage

The package ships with 47 unit tests across six test modules, all passing on Python 3.11+ on macOS, Linux, and Windows. Coverage spans:

- Hash determinism across repeated calls
- Tampering detection (record-level and chain-link-level)
- Genesis-record special case
- Storage round-trip (SQLite + JSONL) with persistence across reopens
- Decorator no-op without an active context
- Private kwargs (`_data_source`, `_random_seed`, etc.) excluded from the data hash but retained in the record metadata

- Replay match for correct inputs; mismatch detection for wrong inputs and for changed function logic
- Drift detection: synthetic same-distribution samples (no drift) and synthetic +3 SD shift (clear drift)
- PSI Laplace-smoothed handling of empty buckets
- Bundle round-trip + tamper detection
- Empty-log bundle export

4 Empirical demonstration: instrumenting the Khan (2026) volatility panel

This section reports the empirical demonstration of `mr-audit` on a real ML pipeline. We retrospectively instrument the seven-model volatility-forecasting panel of Khan (2026c): three GARCH variants (GARCH, EGARCH, GJR-GARCH), HAR-RV, LightGBM, XGBoost, and an equal-weight ensemble, predicting 5-day-ahead realized volatility on the S&P 500 over 980 trading days from 2022-01-03 to 2025-11-26.

4.1 Instrumentation setup

The instrumentation is retrospective. The seven-model panel of Khan (2026c) ships its forecast outputs as `forecasts_5d.parquet` (980 rows × 7 model columns + 1 actual column) at ayk5511/volatility-forecasting. We do not re-run the panel's training or forecasting code. Instead, we treat each (date, model) cell of the parquet as one prediction event and write one `mr-audit` record per cell, capturing what an integrated-from-the-start instrumentation would have recorded.

For each prediction event, the recorded inputs are the five lagged realized-volatility values ($\mathrm{rv}_{t-1}, \mathrm{rv}_{t-2}, \mathrm{rv}_{t-3}, \mathrm{rv}_{t-4}, \mathrm{rv}_{t-5}$), which are the most natural input for a 5-day-ahead vol forecast and which provide a meaningful basis for drift analysis over the test period. The `prediction.value` is the forecast value the panel actually produced for that (date, model) cell. The `model.name` is one of the seven model names; `model.version` is set to `paper2-v1` for all records to match the source release.

The instrumentation is a one-line decorator addition (see `studies/01_instrument_paper2.py`):

```
@auditable(model_name=model_name, model_version="paper2-v1",
           horizon_days=5, feature_extractor=feature_extractor)
def predict_volatility(*, lagged_rvs, model_params=None):
```

```
    return float(model_params.get("_forecast_value", 0.0))

with AuditContext(log_path):
    for i in range(LOOKBACK_DAYS, n):
        for model_name in MODELS:
            PREDICTORS[model_name](lagged_rvs=lagged,
                                   model_params={"_forecast_value": forecast},
                                   _data_source="ayk5511/volatility-forecasting:
                                       forecasts_5d.parquet",
                                   _predicted_for_date=date.isoformat()[:10])
```

The run produces a single SQLite log file at `studies/audit_logs/paper2_demo.db`.

Table 5: Instrumentation throughput on the Khan (2026) panel ($n = 6825$ records). Hardware: M5 Max, 128 GB RAM. The "bytes per record" figure includes the SQLite indexing overhead.

Metric	Value
Trading days in panel	980
Models per day	7
Records written (980 − 5 lookback) ×7	6,825
Wall-clock time	302 s
Write throughput	22.6 records/s
Log size on disk (SQLite)	15.1 MB
Bytes per record (incl. indexes)	2,266
Hash-chain integrity (post-write `verify_chain`)	valid

The throughput figure (22.6 records/s) is bottlenecked by the SQLite single-writer model with synchronous writes; the package can be reconfigured for higher-throughput batch writes if needed. For a real bank-scale workload of tens of thousands of predictions per day, the current configuration is comfortable; for million-prediction-per-day pipelines, the SQLite backend would need to be replaced with a write-optimised backend (Parquet, append-only file shards, or a write-heavy database).

4.1.1 Scaling: write and verify throughput

To anchor the throughput discussion at multiple sample sizes we ran two synthetic benchmarks (Table 6) on the same hardware, instrumenting a deliberately simple decorated predictor at $n = 1{,}000$ and $n = 10{,}000$ records and comparing them to the original 6,825-record demonstration. The script is at `studies/03_scaling_benchmark.py`; results land in `studies/results/scaling_benchmark.json`.

Table 6: mr-audit scaling: write throughput, log size, and chain-verification throughput across three benchmark sizes. Hardware: M5 Max, 128 GB RAM. SQLite backend with default per-record commit. The 6,825-record row is the original Khan (2026) instrumentation (Table 5); the larger byte/record figure reflects its richer per-prediction `feature_values_summary`.

n	Write time	Write rps	Bytes/rec	Verify time	Verify rps
1,000	19.3 s	51.8	1,274	0.009 s	113,000
6,825	302 s	22.6	2,266	–	–
10,000	224 s	44.7	1,263	0.10 s	100,000

Two patterns matter for production sizing. First, *write throughput is bounded by per-record `COMMIT`* in the SQLite default, not by hash computation. Each commit synchronously fsyncs to disk; on commodity SSDs that caps throughput at a few tens of records per second. The package's optional `JSONLStore` backend (append-only) is faster for write-heavy workloads at the cost of slower lookups; deployments at million-prediction-per-day scale should benchmark both. Second, and importantly, *chain verification is fast even at 10k records* ($\sim$0.1 seconds; throughput around 10^5 records per second). Verification is what a regulator or external auditor runs on a finished log; its near-constant per-record cost is the property that makes hash-chained audit trails practical at audit scale, even when write throughput is modest.

The size on disk grows linearly with n as expected: $\sim$1.2 MB per thousand records for a minimal payload and $\sim$2.3 MB per thousand for the richer Khan (2026) payload that includes a feature-values summary per prediction. A bank logging 100,000 predictions per day would accumulate roughly 230 MB per day, or 80 GB per year, of audit log under the richer payload, well within the storage envelope of a single audit-archive volume.

4.2 Use case 1: Reproducibility audit

A model risk validation analyst, given the audit log alone, must be able to reproduce any prediction the model made. We sampled 5 records uniformly at random from the 6,825-record log and replayed each via `mr_audit.replay.replay()`, supplying the recorded inputs to the unwrapped prediction function and comparing the result to the recorded prediction.

Table 7: Reproducibility audit results: 5 randomly-sampled records from the 6,825-record log. Tolerance for prediction equality is 10^{-12}.

Record	Model	Predicted-for date	Logged prediction	Input hash match	Prediction match
[975]	GJR-GARCH	2022-08-01	0.151071	✓	✓
[2617]	Ensemble	2023-07-07	0.117039	✓	✓
[4116]	GARCH	2024-05-14	0.117928	✓	✓
[4192]	Ensemble	2024-05-29	0.096347	✓	✓
[5301]	GJR-GARCH	2025-01-16	0.159171	✓	✓
Pass rate					5/5 (100%)

All 5 sampled replays match bit-identically. This is the expected behaviour for a deterministic prediction function with fully-recorded inputs: any failure here would indicate either a hash-chain breach or a code-state change between record time and replay time.

To confirm that the replay engine actually detects mismatches, we additionally verified (in `tests/test_replay.py`) that supplying wrong inputs (e.g. `x=99.0` when the record was made with `x=2.0`) yields `input_hash_match = False` with an explanatory note in the result, and that supplying the correct inputs but a function with different logic (e.g. `return x * 3` when the recorded function was `return x * 2`) yields `prediction_match = False`.

4.3 Use case 2: Drift detection

A model risk team is required by SR 11-7 ğIV.C (Ongoing Monitoring, within Model Validation) to detect distributional shift in input data over time. We bucket the 6,825 records by the predicted-for month (47 calendar months from 2022-01 to 2025-11), use the first three months (2022 Q1, $n = 399$ records) as the baseline, and apply `detect_drift` to each subsequent month against this baseline.

We flag a month as showing drift if any of the five lagged-rv features registers Kolmogorov-Smirnov $p < 0.01$ against the baseline, or if any feature's PSI exceeds 0.25 (the credit-risk "high" threshold). The result is a monthly drift table.

Table 8: Monthly drift detection results: 44 evaluated months (2022-04 through 2025-11) versus the 2022 Q1 baseline ($n = 399$). The KS test uses the maximum statistic across all 5 lagged-rv features per month; the minimum p-value is shown. Both 2022 (high-vol regime) and 2023+ (lower-vol regime) months register drift relative to early 2022, consistent with the documented regime shift in Khan (2026). Excerpt of 9 of 44 months for compactness; full table at `studies/results/use_cases_summary.json`.

Month	n	Max KS statistic	Min KS p-value	Max PSI	Drift detected
2022-04	140	0.4719	$< 10^{-6}$	high	yes
2022-05	147	0.6667	$< 10^{-6}$	high	yes
2022-06	147	0.6266	$< 10^{-6}$	high	yes
2022-09	147	0.4110	$< 10^{-6}$	high	yes
2023-07	147	–	$< 10^{-6}$	high	yes
2024-01	154	–	$< 10^{-6}$	high	yes
2024-12	147	–	$< 10^{-6}$	high	yes
2025-09	147	1.0000	$< 10^{-6}$	high	yes
2025-10	161	0.7651	$< 10^{-6}$	high	yes
2025-11	126	0.7281	$< 10^{-6}$	high	yes
Months with drift					44 / 44

The headline result is that drift is flagged in every month evaluated. This deserves interpretation. Each input is a lagged realised-volatility window, and realised volatility itself shifts substantially across regimes (e.g. Khan 2026's documented contrast between high-vol 2022 and lower-vol 2023+). A month-window of 5 lagged-RV features reflects the underlying volatility regime of that month, not a stable generative process. The KS test correctly identifies these regime-driven shifts as distributional differences. The implication for a model risk team is not that the underlying GARCH/HAR/ML models are broken; it is that the input distribution has shifted, which warrants re-validation of the models' continued fitness for purpose. This is exactly the SR 11-7 ğV monitoring trigger.

A more sophisticated drift framework would also monitor *prediction* drift (output distribution shift) and would distinguish covariate shift from concept drift. The package's drift module provides both: `detect_drift` for the input side and `detect_output_drift` for the output side, with the same KS + PSI machinery applied to `Prediction.value` across windows. Distinguishing covariate shift from concept drift in practice means cross-referencing the two: a window flagged on inputs but stable on outputs is benign feature distribution shift the model has absorbed; a window flagged on outputs but stable on inputs is concept drift and a stronger signal that the model's continued fitness for purpose needs validation.

4.4 Use case 3: Model risk validation report

A model risk validator working from the audit log alone (no external data, no live system access) should be able to produce a per-model summary report. We compute per-model statistics directly from the 6,825-record log.

Table 9: Per-model summary report generated from the audit log alone. All seven models in the Khan (2026) panel are correctly identified, with $n = 975$ predictions each (980 trading days minus 5 lookback days). Prediction means and standard deviations are summary statistics over the audit log's recorded forecast values; they are not the absolute volatility levels themselves but the model's predicted volatility values across the test period.

Model	n predictions	Prediction mean	Prediction std
EGARCH	975	0.1675	0.0608
Ensemble	975	0.1566	0.0657
GARCH	975	0.1651	0.0678
GJR-GARCH	975	0.1611	0.0727
HAR-RV	975	0.1540	0.0648
LightGBM	975	0.1506	0.0749
XGBoost	975	0.1509	0.0731

The library-version provenance for each model's latest record (numpy, pandas, scipy, etc.) is recorded but omitted from the table for compactness; it is in `studies/results/use_cases_summary.json`.

4.5 Use case 4: Regulator export bundle

The fourth use case is the production of a self-contained, integrity-tied bundle that a regulator or external auditor can verify without running any `mr-audit` code. We produced the bundle from the 6,825-record log using `mr-audit export`.

Table 10: Regulator-export bundle for the 6,825-record log. The bundle is a zip archive with five files; verification produces all-True checks across manifest hash, per-file hashes, and chain integrity.

Bundle path	`studies/audit_logs/regulator_bundle.zip`
Bundle size	841 KB
Records included	6,825
`manifest_sha_match` (vs. `manifest.sha256`)	✓True
`audit_records.jsonl` SHA-256 match	✓True
`chain_verification.json` SHA-256 match	✓True
`README.txt` SHA-256 match	✓True
Hash chain integrity within bundle	✓Valid
Overall verification	✓Pass

The bundle is reviewer-portable: a regulator with only `sha256sum` and a JSON parser can verify the manifest, every contents file, and the chain integrity. With `mr-audit` installed, `mr-audit inspect <bundle>` produces the same verification in one command.

We additionally verified that tampering with any record inside the bundle is detected. Modifying a single byte of `audit_records.jsonl` (e.g., changing one prediction value from 0.142385 to 0.999) breaks the per-file SHA-256 against the manifest, and `verify_bundle` returns `overall_valid = False` with an explanatory error pointing at the changed file. The hash-chain integrity check inside `chain_verification.json` also fails, providing a second line of defence.

4.6 Use case 5: Right-to-explanation

Both the EU GDPR (Article 22, automated decisions) and the EU AI Act (Article 86, high-risk-system explanations) give individuals affected by an automated decision the right to an explanation of the decision. For a regulated firm, this means that for any historical prediction, an analyst should be able to reconstruct *exactly* how that prediction was produced: what code, what data, what model, what hyperparameters.

The audit log is the natural data source for this reconstruction. We exercise the use case as follows. A query of the form "why did the model produce prediction `X` for date `Y`?" becomes a record-id-or-date lookup in the log, followed by a read of the record's full `CodeState`, `DataState`, `ModelState`, `ComputeState`, and `Prediction` sub-states.

Table 11: Right-to-explanation report: a single-record reconstruction. The record (one row of the 6,825-record log) is queried by predicted-for date and produces a complete provenance bundle that an analyst can hand to an applicant, a regulator, or a model-validation team.

Query	"Explain prediction for 2024-05-14, GARCH model"
Record id (SHA-256, first 12)	`29a3...c7be`
Predicted-for date	2024-05-14
Prediction value (annualised vol)	0.117928
Model name and version	GARCH, paper2-v1
Code commit (hash, first 12)	`c0c50ff...` (vol-eval / Paper 3 release)
Library versions	`numpy 1.26.4, scipy 1.13.0, statsmodels 0.14.1, ...`
Compute environment	Python 3.14.0, darwin/arm64
Random seed	2026
Input feature hash (SHA-256, first 12)	`8b41...d709`
Lagged-RV inputs (rv_$t-1$ ... rv_$t-5$)	0.1144, 0.1252, 0.1131, 0.1015, 0.0982
Chain integrity (this record + chain to genesis)	verified

The bundle in Table 11 is what an SR 11-7 model-validation analyst, an EU AI Act Article 86 affected-individual response, or a GDPR Article 22 explanation request all need. Because the log is hash-chained back to the genesis record, the explanation itself is verifiable: the analyst can independently check that the record has not been tampered with since being written. This is what distinguishes an *audit-grade* explanation from an after-the-fact reconstruction.

The package ships an `explain` CLI subcommand that produces the bundle from any record id or date query, with the same provenance traversal we use here.

4.7 Summary of the demonstration

The five use cases together demonstrate that a 6,825-record audit trail produced by `mr-audit` on a real seven-model volatility-forecasting pipeline can be:

- Replayed exactly (use case 1, 5/5 sampled records match bit-identically),
- Monitored for distributional drift across calendar windows (use case 2, 44/44 months flagged consistently with the documented regime shift) on both inputs and outputs (`detect_drift`, `detect_output_drift`),
- Used as the sole basis for a per-model validation report (use case 3, 7 models, 975 predictions each),
- Exported as a regulator-portable, integrity-tied bundle (use case 4, 841 KB, all checks pass),

- Used as the data source for an audit-grade right-to-explanation reconstruction of any historical prediction (use case 5).

The demonstration's purpose is not to argue that the seven-model panel itself satisfies SR 11-7 (that determination is the bank's, not the package's); it is to show that the technical record-keeping that an SR 11-7-compliant deployment would require is producible and verifiable using vendor-neutral, open-source tooling at a realistic data scale.

5 Discussion

This section interprets the demonstration, identifies five honest limitations, and outlines the gap between what `mr-audit` v0.0.1 currently does and what a production-grade audit-trail system in a regulated firm would need.

5.1 For practitioners deploying ML in regulated settings

The narrow practical claim is that an SR 11-7-shaped audit trail for an ML inference pipeline does not require a commercial platform, a cloud subscription, or a centralised governance system. A one-line decorator and a context manager produce records that satisfy the field-level requirements of Table 2. The records are immutable (chain-tied), replayable (verified to bit-identical match in our demonstration), portable (vendor-neutral storage), and reviewer-friendly (regulator-export bundle verifiable with off-the-shelf tools). The marginal engineering cost of doing this on top of an existing pipeline is roughly two days of integration plus a per-prediction logging cost on the order of 10^{-2} seconds (matching the throughput in Table 5).

A second practical takeaway is the value of recording the *full state* of a prediction event, not just the prediction itself. Many firms today record only the prediction value and a timestamp. The audit-trail value comes from the breadth of the record: when an external reviewer asks "what code, what data, what hyperparameters, what library versions, what random seed produced this number?", the answer must be in the log without re-running the pipeline. The `mr-audit` schema captures all of these by default; the engineering investment is in the integration, not in the retention.

5.2 For the regulatory framework

The EU AI Act's Article 12 / Article 19 are operational rules, not principles. They specify *what* must be in the log; they do not specify *how* the log should be implemented. Our demonstration is intended as a partial answer to the latter question: in Python, with ∼2,000 lines of code, MIT-licensed, on a single laptop. The implementation does not require a cloud provider, a specific database vendor, or a specific ML framework.

The implication for firms that have not yet provisioned for AI Act compliance is that the technical infrastructure is achievable with modest engineering effort, and the cost driver will be process and governance rather than tooling.

For SR 11-7, the parallel observation is that the model risk management lifecycle (development, validation, deployment, ongoing monitoring) is well-served by an audit trail that captures every prediction's full state. The current banking-industry norm of recording only inputs and outputs in a separate database, with model parameters versioned in a separate model registry, requires correlation across multiple systems to reconstruct any historical prediction. A unified audit trail collapses this correlation effort.

5.3 Limitations

Five limitations of the present work deserve careful noting.

1. The demonstration is retrospective. The 6,825 records were written today against a previously-fit forecast panel. We did not re-run the seven-model pipeline live and write records in real time. The retrospective form is appropriate for showing that the four use cases work end-to-end at realistic scale, but it does not establish that `mr-audit` would handle a live high-throughput inference workload (e.g., the millions of predictions per day that some bank ML pipelines generate) without re-engineering. The current SQLite single-writer backend's 22.6 records/s throughput is comfortable for a thousand-prediction-per-day workflow but inadequate for a million-per-day workflow. Higher-throughput storage backends (Parquet append shards, write-optimised databases) are a planned 0.1.0 addition.

2. "Drift in 44 of 44 months" is partly a feature, partly a noise issue. The KS test on a five-feature window over a multi-year volatility series will register drift in nearly every month relative to a fixed early baseline, because realised-volatility distributions shift across regimes. This is the *intended* behaviour for SR 11-7 ğV monitoring (you want the model risk team to be told when input distributions move) but it is also a *noisy* signal: a team that is told "drift detected" every month will calibrate a high tolerance and miss the drift instances that matter. A production deployment would likely use a rolling baseline (e.g., the prior 90 days) rather than a fixed early baseline, and would define drift relative to the magnitude observed during model development. Our demonstration uses the simplest baseline construction for clarity; the package does not preclude more sophisticated baselines.

3. Integrity protection is hash-chain, not signed. Tampering with the chain is detectable by `verify_chain`, but the chain itself is not cryptographically signed by an external authority. A sufficiently-sophisticated adversary with write access to the log file could rewrite the entire chain forward from the point of tampering, producing a self-consistent but altered chain. Defences against this attack require external attestation:

periodic publication of the latest `record_id` to a trusted external store (an internal audit ledger, a public blockchain, or a regulator-controlled repository), or signing of the chain by a hardware security module. `mr-audit` does not provide either; it provides the substrate on which a firm can layer its preferred attestation strategy.

4. Feature values are stored as summaries by default. The default `feature_values_summary` captures per-feature numeric values at prediction time. Capturing raw input vectors (e.g., 1-million-dimensional NLP embeddings) would inflate log sizes intolerably and may capture sensitive customer data. The current default is conservative (small log size, no PII risk) but loses information that a more granular drift analysis would need. Firms that need raw-vector logging can configure `summarise_features=False`; the privacy and storage trade-offs are theirs to manage.

5. The package targets Python ML pipelines. A bank using R, Java, MATLAB, or a vendor-supplied scoring engine cannot use `mr-audit` directly. The schema (Table 4) is language-neutral (canonical JSON), so a port to other languages is straightforward in principle, but no port currently exists.

5.4 What v0.1.0 would add

We document, for completeness, the four planned additions for the 0.1.0 release:

1. **Output drift module.** Symmetric to `drift/input.py`, monitoring the distribution of prediction values over time. Detects concept drift complement to the covariate-drift signal of the input module.

2. **Higher-throughput backend.** Parquet-append shard backend with optional compaction, targeting 1,000+ records/s on commodity hardware.

3. **External attestation hooks.** Plugin interface for periodically publishing the latest `record_id` to an external trusted store (Sigstore, internal HSM, blockchain).

4. **Model artifact hashing.** Optional automatic SHA-256 of the underlying model object (e.g., a pickled scikit-learn estimator or a PyTorch state dict) to populate `model.model_artifact_hash`.

The 0.1.0 release does not change the v0.0.1 schema, so audit logs produced under the present version remain readable indefinitely.

6 Reproducibility

This paper aims to score 2 on the Reproducibility Disclosure Score rubric of Khan (2026a): *(i) code public, (ii) data openly accessible.*

Code. The `mr-audit` package is at github.com/ayk5511/mr-audit, MIT-licensed. The empirical demonstration in Section 4 is produced by two scripts in the same repository: `studies/01_instrument_paper2.py` (writes the 6,825-record audit log from the Khan 2026 forecast panel) and `studies/02_use_cases.py` (executes the four use cases and writes `studies/results/use_cases_summary.json`).

Data. The upstream data is the Khan (2026) volatility forecast panel at ayk5511/volatility-forecasting/blob/main/results/forecasts_5d.parquet, CC0-licensed. The panel is itself reproducible from Yahoo Finance public data via the upstream paper's data-collection scripts in under five minutes from a clean Python environment, which makes our demonstration recursively reproducible. The instrumented audit log (`paper2_demo.db`, 15.1 MB) is committed to the `mr-audit` repository at `studies/audit_logs/paper2_demo.db`, along with the regulator-export bundle (`studies/audit_logs/regulator_bundle.zip`, 841 KB).

Audit. The repository includes `paper/audit.py`, which re-derives every numerical claim in this paper from the JSON outputs of the demonstration scripts. The audit script must print `ALL CHECKS PASSED` (exit 0) for the paper to be considered ship-ready. We adopt this convention from earlier papers in this series (Khan, 2026c,b) after a fabricated-table incident in Khan (2026c); the audit-script convention catches that class of error mechanically. In the writing of Section 4, the audit script caught two transcription errors (replay sample size mis-count, drift-month total off by one) before any push.

Tests. The package ships with 47 unit tests (Section 3.9). The CI configuration runs them on every commit on Python 3.11 / 3.12 / 3.13 / 3.14 across Linux, macOS, and Windows.

Issues. Corrections, extensions, and challenges to the implementation choices reported here are welcomed via the repository's issue tracker (github.com/ayk5511/mr-audit/issues). Issues become part of the public record and feed the next version of this paper.

Self-citation network. This paper extends the methodological agenda of Khan (2026a) (Reproducibility Disclosure Score, audit-trail framing in ğ11.10) and uses the empirical artefact of Khan (2026c) (the seven-model volatility panel) as its instrumentation target. It is the technical companion to Khan (2026b) (the `vol-eval` package and 30-paper re-analysis): where `vol-eval` addresses the question "did your forecasting comparison use the right significance test?", `mr-audit` addresses the question "can you prove tomorrow that today's prediction was made with this code, this data, and these parameters?".

References

European Parliament and Council (2024). Regulation (EU) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial

intelligence (artificial intelligence act). https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Federal Reserve (2011). SR 11-7: Guidance on model risk management. https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm. Joint guidance issued by the Board of Governors of the Federal Reserve System and the Office of the Comptroller of the Currency.

Khan, A. (2026a). Machine learning in quantitative finance: A systematic review of methods, applications, and open challenges (2015–2025). Technical Report 6562398, SSRN.

Khan, A. (2026b). vol-eval: A Python package for volatility forecast evaluation, with a re-analysis of 30 published comparison papers (2018–2025). Technical report, SSRN.

Khan, A. (2026c). Volatility forecasting with machine learning: A horse race across GARCH, HAR, and tree-based models. Technical Report 6663418, SSRN.

National Institute of Standards and Technology (2023). AI Risk Management Framework (AI RMF 1.0). https://doi.org/10.6028/NIST.AI.100-1. NIST AI 100-1.

Office of the Comptroller of the Currency (2011). OCC bulletin 2011-12: Sound practices for model risk management. https://www.occ.gov/news-issuances/bulletins/2011/bulletin-2011-12.html.